

RESUMO DO MÓDULO

Route Handlers

Os Route Handlers (ou manipuladores de rota) são funções que lidam com as solicitações para rotas específicas em um aplicativo Express. Eles são responsáveis por processar a requisição do cliente e enviar uma resposta de volta.

Cada rota no Express pode ter um ou mais manipuladores de rota, que são executados em uma ordem específica. Isso permite que você imponha pré-condições ou realize verificações em uma rota antes de permitir que ela seja totalmente executada.

Diferentes Formas de Criar Route Handlers no Express

Existem várias maneiras de definir e organizar os manipuladores de rota no Express. Vamos explorar algumas das abordagens mais comuns e entender quando utilizá-las.

1. Definindo um Route Handler Simples

A maneira mais simples de definir um manipulador de rota é criar uma função anônima diretamente na rota:

```
const express = require('express');
```

```
const app = express();
```

```
app.get('/', (req, res) => {
```

```
  res.send('Olá, mundo!');
```

```
});
```

```
app.listen(3000, () => {
```

```
  console.log('Servidor iniciado em http://localhost:3000');
```

```
});
```

Essa abordagem é ideal para projetos pequenos, mas em aplicações maiores pode tornar o código confuso e difícil de manter.

2. Usando Funções Nomeadas como Manipuladores de Rota

Uma maneira mais organizada é definir a função manipuladora separadamente e, em seguida, associá-la à rota. Isso torna o código mais legível e fácil de manter.

```
const express = require('express');

const app = express();

const mostrarOlaMundo = (req, res) => {
  res.send('Olá, mundo!');
};

app.get('/', mostrarOlaMundo);

app.listen(3000, () => {
  console.log('Servidor iniciado em http://localhost:3000');
});
```

3. Usando Vários Route Handlers com next()

O Express permite que você defina vários manipuladores de rota para a mesma rota, executando-os em sequência usando `next()`. Isso é útil para aplicar lógica de pré-condição e realizar ações intermediárias antes de enviar a resposta final.

```
app.get(
  '/',
  (req, res, next) => {
```

```
    console.log('Olá, mundo!');

    next(); // Passa o controle para o próximo manipulador de rota
  },

  (req, res) => {

    res.send('Este é o segundo route handler');

  }

);
```

Nesse exemplo:

O primeiro manipulador exibe a mensagem "Olá, mundo!" e chama `next()` para passar o controle para o próximo manipulador.

O segundo manipulador envia a resposta "Este é o segundo route handler".

Se `next()` não fosse chamado no primeiro manipulador, o segundo não seria executado.

4. Usando Funções como Argumentos de Rota

Você pode definir funções e passá-las como argumentos para a rota. Isso é útil quando você deseja reutilizar a mesma função em várias rotas.

```
function logRequest(req, res, next) {

  console.log('Nova requisição recebida!');

  next();

}
```

```
app.get('/', logRequest, function(req, res) {

  res.send('Olá, mundo!');

});
```

Nesse exemplo, a função `logRequest` é executada antes de enviar a resposta "Olá, mundo!".

5. Usando um Array de Funções como Route Handlers

Você pode passar um array de funções que serão executadas em sequência, o que permite modularizar ainda mais a lógica da rota.

```
const cb0 = (req, res, next) => {  
  console.log('CB0');  
  next();  
};
```

```
const cb1 = (req, res, next) => {  
  console.log('CB1');  
  next();  
};
```

```
const cb2 = (req, res) => {  
  res.send('CB2');  
};
```

```
app.get('/', [cb0, cb1, cb2]);
```

A sequência de execução será:

CB0

CB1

CB2

Cada função é chamada na ordem em que foi definida no array.

6. Combinando Funções e Arrays de Funções

Você também pode combinar funções e arrays de funções para obter um fluxo de execução mais

complexo.

```
const cb0 = (req, res, next) => {  
  console.log('CB0');  
  next();  
};
```

```
const cb1 = (req, res, next) => {  
  console.log('CB1');  
  next();  
};
```

```
app.get(  
  '/example/d',  
  [cb0, cb1],  
  (req, res, next) => {  
    console.log('A resposta vai ser enviada para a próxima função ...');  
    next();  
  },  
  (req, res) => {  
    res.send('Última parada!');  
  }  
);
```

7. Usando Router com Manipuladores Encadeados

O Router permite encadear manipuladores de rota, definindo vários handlers para uma mesma rota usando diferentes métodos HTTP (get, post, put, etc.).

Crie um arquivo routes.js:

```
const express = require('express');

const router = express.Router();

router

  .get('/', (req, res) => {

    res.send('Lista de livros');

  })

  .post('/', (req, res) => {

    res.send('Novo livro adicionado');

  });

module.exports = router;
```

No arquivo principal (app.js):

javascript

```
const express = require('express');

const app = express();

const router = require('./routes.js');

app.use('/', router);

app.listen(3000, () => {

  console.log('Servidor iniciado em http://localhost:3000');

});
```

Quando Usar Cada Abordagem?

A escolha do tipo de manipulador de rota depende do tamanho e da complexidade da aplicação:

Funções anônimas diretamente na rota: Usado em projetos pequenos e exemplos simples.

Funções nomeadas: Facilita a leitura e reutilização do código.

Vários manipuladores com `next()`: Ideal para aplicar lógica intermediária, como validação e autenticação.

Arrays de funções: Modularizam o código e facilitam a execução sequencial de lógica.

Combinação de Router e manipuladores encadeados: Melhor para projetos maiores, onde cada grupo de rotas pode ser organizado em um módulo separado.

Conclusão

Os Route Handlers são uma parte fundamental do desenvolvimento com Express. Eles permitem modularizar a lógica de tratamento das rotas e aplicar diferentes abordagens, como múltiplos handlers, funções nomeadas e arrays de funções. Escolher o tipo correto de manipulador depende das necessidades específicas do projeto e da clareza e manutenção do código.
